11 — Mapa entre o texto do TCC e o código

> Código: `src/mpu.cpp`, `src/sensors.cpp`, `src/motor.cpp`, `src/settings.cpp` · Relacionados: 01 — Visão geral, 10 — Decisões

Liga cada afirmação técnica do texto do TCC ("Colete Ortopédico Inteligente para Monitoramento Postural em Tempo Real", versão de 19/10/2026) ao ponto do código que a implementa, lista o que precisa ser corrigido no texto e sugere o que falta escrever sobre o software.

1. Afirmações do texto e onde estão no código

Seção do texto	Afirmação	No código	Situação
Resumo	MPU-6050 com acelerômetro e giroscópio num módulo	`src/mpu.cpp` (driver), `src/sensors.cpp`	ok
Resumo	ESP32 faz aquisição, tratamento e transmissão	`sensores_atualizar`, `TiltFilter`, `DetectorPostura`, `web_enviarDados`	ok
1	Sensores de flexão (plural) para a curvatura	Flex 1 (GPIO34) e flex 2 opcional (GPIO35, desligado por padrão)	ok; ver 05
1	Motor vibratório acionado quando há desvio	`motor_alerta` em `atualizarPostura`	ok
1.1	Resistência do flex aumenta ao flexionar; leitura por divisor de tensão	Divisor 3V3–flex–GPIO34–10 kΩ–GND; leitura **cai** ao dobrar; `flexPct`	ok
1.1	Resistência inicial 10 kΩ, tolerância ±30%	Curvatura relativa à calibração; faixa válida ampla (100–4000)	ok: a tolerância é o motivo da medida relativa
1.2	Sensibilidade 131 LSB/°/s e 16384 LSB/g	`LSB_POR_DPS = 131`, `LSB_POR_G = 16384` em `src/mpu.cpp`	ok
1.2	Escalas ±250 °/s e ±2 g	`GYRO_CONFIG = 0x00`, `ACCEL_CONFIG = 0x00` em `Mpu6050::iniciar`	ok
1.2	"fator variando de um a quatro"	—	**corrigir** (item 2)
1.3	ADC de 12 bits para o flex	`analogReadResolution(12)`, só ADC1	ok
1.3	I2C com o MPU-6050	`Wire` a 100 kHz, GPIO21/22	ok
1.3	Wi-Fi e Bluetooth	Wi-Fi em Access Point; Bluetooth **não usado**	complementar
1.3	Flash 0.512KB, EEPROM 428KB	—	**corrigir** (item 1)
1.4	Motor 3 V, 60 mA, vibração contínua (DC)	Motor DC acionado por **PWM 20 kHz** via transistor, alerta em **pulsos** 300/700 ms	complementar

2. Correções sugeridas no texto

1. **Memória do ESP32 (1.3).** "Flash 0.512KB / EEPROM 428KB" está errado. O ESP32-WROOM-32 tem **4 MB de flash** e **não tem EEPROM**: dados permanentes ficam na NVS, uma área da flash (é onde o colete guarda limites e calibração). SRAM ~520 kB está certo.
2. **Faixas do MPU-6050 (1.2).** São quatro faixas: ±250, ±500, ±1000, ±2000 °/s e ±2, ±4, ±8, ±16 g. O fator vai de **1 a 8** e a sensibilidade **divide** por ele (131 → 65,5 → 32,8 → 16,4 LSB/°/s; 16384 → 8192 → 4096 → 2048 LSB/g). O projeto usa a faixa mais sensível (fator 1).
3. **Motor (1.4).** Incluir que o GPIO não fornece 60 mA e por isso há um transistor NPN com resistor de base de 1 kΩ e diodo de proteção; e que o firmware usa PWM e pulsos (ver 06).
4. **Numeração das figuras.** Há duas "Figura 2" e duas "Figura 3".

5. **Itens que faltam no hardware:** botão de calibração, transistor + diodo, capacitor de 100 µF, power bank, esquema de ligação (figura do `montagem.pdf`).

3. O que escrever sobre o software (roteiro sugerido)

Seção sugerida	Conteúdo	Base nesta documentação
Arquitetura do sistema	Colete autônomo + app web embarcado; módulos; duas tarefas	01, 02
Aquisição e filtragem	Configuração do MPU, offset do giroscópio, filtro complementar sobre o vetor gravidade, desvio	03
Sensores de flexão	Divisor, média móvel, detecção automática, curvatura relativa	05
Calibração e detecção	Máquina de estados ok/atenção/alerta, histerese, tempo mínimo, calibração com recusa	04
Alerta e persistência	Circuito do motor, PWM, padrões de vibração, NVS	06
Interface com o usuário	Botão, serial, app web (Access Point, WebSocket, gráfico, 3D)	07, 08
Testes e validação	Testes unitários, simulador, roteiro no hardware	09, `VALIDACAO.md`
Resultados	Gráfico desvio × tempo com limite e alerta, vídeo, tabela de validação	`tools/grafico_serial.py`, CSV do app

Frase de resumo para o texto (do plano original): "Aplicação web embarcada, servida pelo próprio microcontrolador, com comunicação bidirecional via WebSocket sobre Wi-Fi em modo Access Point."

Figuras sugeridas

Figura	Diagrama
Contexto do sistema	01, seção 2
Ligações elétricas	01, seção 3 ou o esquema do `montagem.pdf`
Arquitetura em camadas	02, seção 2
Tarefas e comunicação entre elas	02, seção 3
Fluxo de dados do sensor ao alerta	02, seção 5
Cálculo da inclinação	03, seção 1
Máquina de estados da postura	04, seção 2
Sequência da calibração	04, seção 3
Protocolo app ↔ colete	07
Resultado: desvio × tempo	PNG gerado por `tools/grafico_serial.py`

Como exportar os diagramas em imagem: README.md.

4. Perguntas prováveis da banca

Como o colete sabe qual é a postura correta? Pela calibração: a pessoa veste, fica na postura correta e aperta o botão. O colete grava a direção da gravidade naquele instante (média de 1 s) e passa a medir o desvio em relação a ela. Sem calibração ele não vibra.

Por que usar giroscópio e acelerômetro juntos? O acelerômetro dá a direção absoluta da gravidade, mas é ruidoso e sente os movimentos; o giroscópio é suave, mas acumula erro. O filtro complementar usa o giroscópio para os movimentos rápidos e o acelerômetro para corrigir a deriva (98% / 2% a cada 5 ms, constante de tempo

de ~0,25 s).

Por que não calcular pitch e roll, como é comum? As fórmulas de pitch/roll supõem o sensor deitado. Em pé, nas costas, uma delas passa por uma singularidade e o ângulo salta ~20° com 4° de movimento real. Medir o ângulo entre o vetor gravidade atual e o da calibração não tem esse problema e não depende de como o sensor foi preso.

O que é a histerese e por que usar? É uma margem para sair do alerta: entra acima de 20° e só sai abaixo de 17°. Sem ela, um desvio oscilando em torno de 20° ligaria e desligaria o motor sem parar.

Por que esperar 3 segundos para vibrar? Para não vibrar em movimentos rápidos e normais (pegar algo no chão). Só a postura ruim que se mantém gera alerta. O tempo é ajustável.

E se o sensor flex quebrar ou soltar? No modo padrão (`auto`), o firmware percebe (leitura fora da faixa ou instável por 1 s), tira o flex da decisão e segue só com o MPU. Se o flex tiver sido forçado ligado (`set flex on`), ele continua na decisão e um flex solto dispara o alerta; por isso o modo recomendado é `auto`.

E se o celular desconectar? Nada muda no colete: a detecção e o motor rodam numa tarefa separada, de prioridade maior que a do `loop()` (onde rodam a página, o WebSocket e a serial), e não dependem do app.

Os ajustes se perdem ao desligar? Não. Ficam na NVS, uma área da flash do ESP32.

Como foi testado sem o hardware? A lógica (inclinação, filtro, detecção, calibração, filtros dos sensores) tem 21 testes automáticos que rodam no PC. O firmware inteiro também roda no PC com um MPU-6050 simulado, seguindo roteiros (inclinar, dobrar o flex, apertar o botão, desconectar o sensor) e ligado à página web real.

Qual a frequência de amostragem? MPU a 200 Hz, flex a 100 Hz, decisão de postura a 50 Hz, envio ao app a 10 Hz.

Qual a precisão da medida? A resolução do acelerômetro é 1/16384 g. A estabilidade do desvio parado e a resposta a movimentos rápidos são medidas no teste com o hardware (`VALIDACAO.md`, V2.4 e V2.6); use os números medidos no texto.

A rede do colete é segura? É uma rede local com senha (WPA2), sem internet. Quem tiver a senha pode ver os dados e mudar os ajustes. Para um protótipo de TCC é suficiente; uma versão comercial precisaria de senha por aparelho.